

PyCon Busan 2026



도메인은 바뀌어도 데이터는 남는다

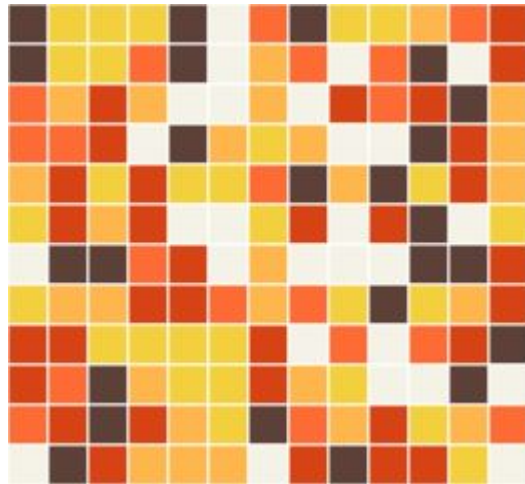
Python으로 문제 정의하는 법

Kyung Seo Kim - 김경서

겉보기에 다른 데이터

바이오

유전자 발현 데이터
행렬 형태, 수만 개의 피처



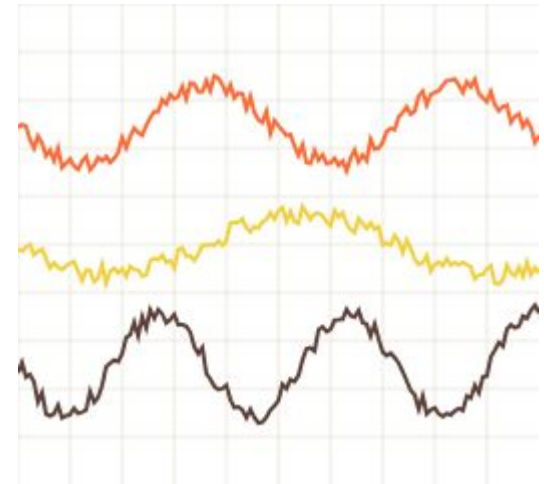
금융

시계열 거래 데이터
정형 테이블 형태

Date	ID	Amount	Status
26/04/01	C1001	\$126	OK
26/04/02	C1002	\$807	PND
26/04/03	C1003	\$314	PND
26/04/04	C1004	\$485	OK
26/04/05	C1005	\$573	OK
26/04/06	C1006	\$791	PND
26/04/07	C1007	\$646	PND

제조

센서 파형 데이터
시간에 따른 센서 기록



도메인을 이해하면, 부딪히는 비슷한 문제

공통 이슈

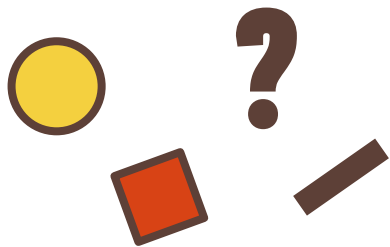
EDA

Modeling

Deployment

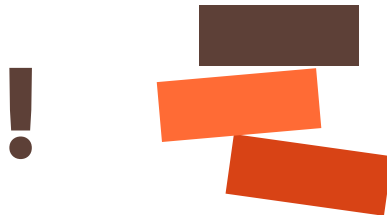
1

피처 정의 기준 부재



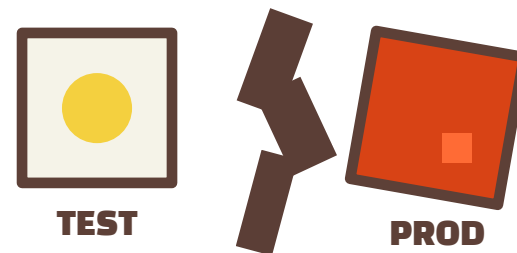
2

데이터 신뢰성 한계



3

테스트-운영 괴리



복잡한 모델보다 좋은 문제 정의가 중요합니다

복잡한 모델 VS 좋은 문제인식

Complex Model

최신 모델과 이론의 지속적 업데이트
기술적 생산성과 심도 깊은 연구 지향
하지만 모든 데이터의 정답은 아님

모델 < 문제 정의

Good Problem

데이터를 보는 단위를 새롭게 정의
가정과 제약 조건의 명확한 이해
해결방식의 흐름을 주도하는 추천 구성

데이터 문제 해결 3단계 프레임워크

1

2

3

데이터 구조 정의

데이터의 단위 확인

- 바이오: 유전자 단위
- 금융: 기업 또는 거래 단위
- 제조: 센서 구간 단위

제약 조건 정의

정의된 구조의 한계점 파악

- Batch effect 존재 여부
- Data leakage 가능성 점검
- 외부 요인 영향도 평가 및
신규 수집 필요성 결정

일반화 가능성 검증

신규 데이터 작동 여부 확인

- 과거 데이터 학습 모델의
테스트/신규 데이터 검증
- 재현성 확보 여부 타진
- 운영 환경 적합성 판단

바이오 | NCBI GEO와 batch effect

NCBI

Resources

How To

Sign in to NCBI

GEO Home

Documentation

Query & Browse

Email GEO

Gene Expression Omnibus

GEO is a public functional genomics data repository supporting MIAME-compliant data submissions. Array- and sequence-based data are accepted. Tools are provided to help users query and download experiments and curated gene expression profiles.

Search

Getting Started

Overview

FAQ

About GEO DataSets

About GEO Profiles

About GEO2R Analysis

How to Construct a Query

How to Download Data

Tools

Search for Studies at GEO DataSets

Search for Gene Expression at GEO Profiles

Search GEO Documentation

Analyze a Study with GEO2R

Studies with Genome Data Viewer Tracks

Programmatic Access

FTP Site

ENCODE Data Listings and Tracks

Browse Content

Repository Browser

DataSets:

4348

Series:

283516

Platforms:

28369

Samples:

8485291

Information for Submitters

Login to Submit

Submission Guidelines

Update Guidelines

MIAME Standards

Citing and Linking to GEO

Guidelines for Reviewers

GEO Publications

방대한 공개 데이터베이스

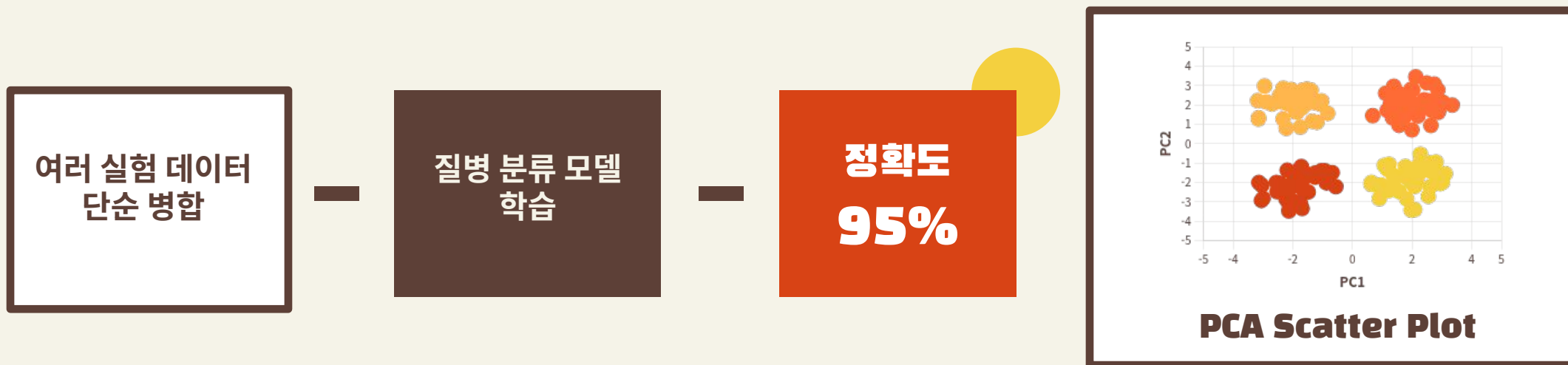
전 세계 연구자들이 올린
유전자 발현 데이터가
수만 건 쌓여 있는 상태

가장 흔하고 치명적인 문제

같은 질병 데이터라도
조건에 따른 결과의 차이

'batch effect'

바이오 | 모델이 학습한 것 = 실험 차이



모델이 학습한 것 = 실험 간의 차이 (Batch Effect)

정확도 95%는 환자의 질병을 맞춘 것이 아닌,
어느 연구실에서 나온 데이터인지를 맞춘 결과

즉, 병합 과정에서 데이터 구조를 잘못 정의한 문제

바이오 | gene ~ pathway aggregation

X Before : 개별 유전자 단위

개별 유전자를 feature로 사용

```
# 1. 개별 유전자 발현량 데이터를 로드 (예: 20,000개 이상의 컬럼)  
# Columns: ['Gene_A', 'Gene_B', 'Gene_C', ..., 'Label']  
df = pd.read_csv("gene_expression_data.csv")  
  
# 2. 유전자 개별 값을 그대로 Feature로 사용  
X = df.drop(columns=['Label'])  
y = df['Label']  
  
# 3. 모델 학습 (High Variance, 실험 간 변동에 민감하여 일반화가 어려움)  
model = RandomForestClassifier()  
model.fit(X, y)
```

실험 간 변동이 커서
배치 효과(Batch Effect)에 취약하며 일반화에 실패

O After : 기능적 묶음 (Pathway)

유전자들을 pathway로 집약하고 실험 단위 정규화 적용

```
# 1. 유전자-경로 매핑 정보 정의 (Domain Knowledge 반영)  
pathway_map = {  
    "Cell_Cycle": ["Gene_A", "Gene_D", "Gene_X"],  
    "Immune_Response": ["Gene_B", "Gene_F", "Gene_Y"],  
    "Metabolism": ["Gene_C", "Gene_E", "Gene_Z"]  
}  
  
# 2. Gene -> Pathway 단위로 Aggregation (평균값 계산)  
pathway_features = pd.DataFrame()  
for p_name, genes in pathway_map.items():  
    # 해당 Pathway에 속한 유전자들의 평균을 새로운 피처로 생성  
    pathway_features[p_name] = df[genes].mean(axis=1)  
  
# 3. 집약된 모델 학습 (Low Complexity, 노이즈 감소로 신규 데이터 일반화 성공)  
model.fit(pathway_features, y)
```

단순해진 모델로 신규 실험 데이터에서도 일반화 성공

gene → pathway aggregation

단위를 바꾸니 일반화가 되었다

금융 | B2B 매출채권 연구

IEEE Xplore®

Browse ▾My Settings ▾Help ▾

Institutional Sign In

All

ADVANCED SEARCH

Conferences > 2025 IEEE International Confe... ?

Quantifying SME Credit Risk in Receivables Financing with AI-Based Transaction-Level Data Analytics in a Fintech Context

Publisher: IEEE

Cite This

PDF

Kyung Seo Kim ; Chung Man Park ; Jae Ho Youn ; In Tak Song ; Chung Hee Ye ; Byung Hee Choi All Authors

30

Full

Text Views

Abstract

Document Sections

I. Introduction & Motivation

II. Data & Methodology

Abstract:

Small and medium-sized enterprises (SMEs) remain disproportionately excluded from formal credit markets due to the structural limitations of traditional assessment models that rely on static financial statements, bureau histories, and collateral. To address this gap, we analyze a proprietary, transaction-level dataset of invoice-based receivables financing from South Korea's Fintech platforms, FlowPay and FlowPoint, covering over five years of live B2B financing activity. We present a domain-specific feature engineering pipeline that transforms raw transactions into high-dimensional risk signals, incorporating temporal dynamics, financial interactions,

핀테크 스타트업에서의 연구

모델 기획부터 실험, 컨퍼런스 발표까지의
기업 단위 데이터 연구 경험

B2B 데이터의 특수성

feature 해석을 위한
기업 심사 담당자와의 대면 인터뷰

금융 | 재무 및 비재무 feature 구조화

01

재무 기준 (정량화 용이)

기업을 평가하는 내부기준으로 정량화된 점수화가 용이함 (명확한 수치 기반)

02

비재무 기준 (서술형→정량화)

심사 담당자의 직관과 서술형 평가를 역산하여 정량화 (사내 경험을 토대로 칼럼별 정의 및 코드화 작업 필수)

03

거래 레벨 데이터 복잡성

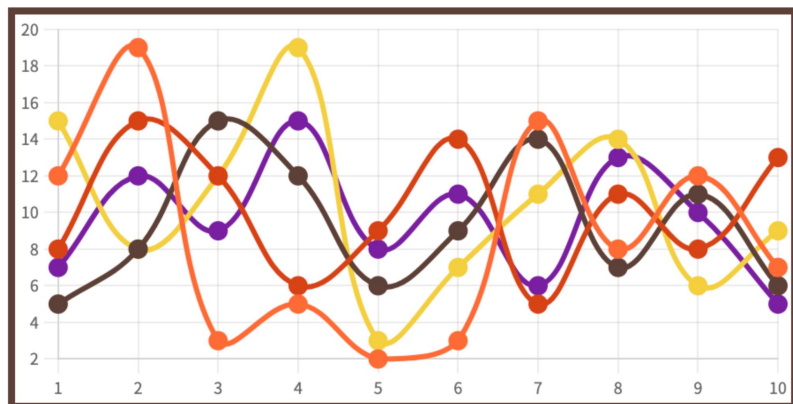
양도, 담보 제공 등 매출채권 흐름이 복잡적 (단순 사업자번호 ID만으로는 독립데이터 샘플링이 어려움)

해결 방향 : "언제의 정보로 판단하는가"를 먼저 정의하고, 기업 단위로 집약

금융 | leakage 방지: 기업 단위 재구성

X Before : 단일 테이블

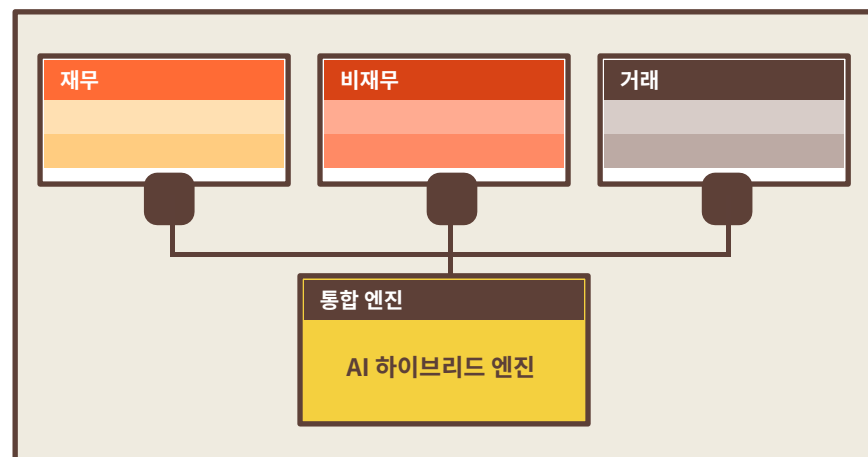
전체 시간축 데이터가 뒤섞여 하나의 테이블로 존재



정밀도는 높지만 단일 모델 학습에 이용하기에는 무의미함

O After : 기업 단위 재구성

feature별 테이블 분리 후 사업자번호 매핑



특성별 모델 적용 및 통합 엔진 구축으로 유의미한 결과 도출

핵심 질문 : "언제의 정보로 판단하는가"

거래 단위 대신 기업 수준 행동 패턴으로 집약

제조 | SECOM 데이터의 현실



01

방대한 590개의 Feature와 결측치

02

불량률 ~2%, 극심한 Class Imbalance

03

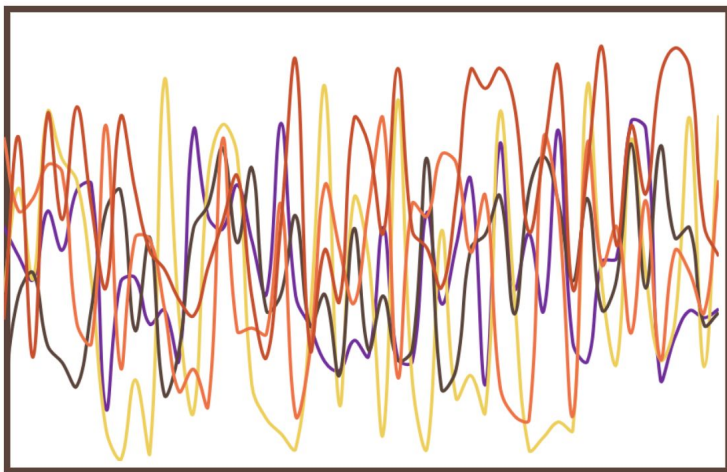
풍부함의 착시와 품질 이슈

고차원 시계열을 그대로 쓰는 것이 아닌, **시간 기반의 공정 상태 문제 정의 필요**

제조 | state abstraction으로 성능 개선

X Before : 고차원 시계열 입력

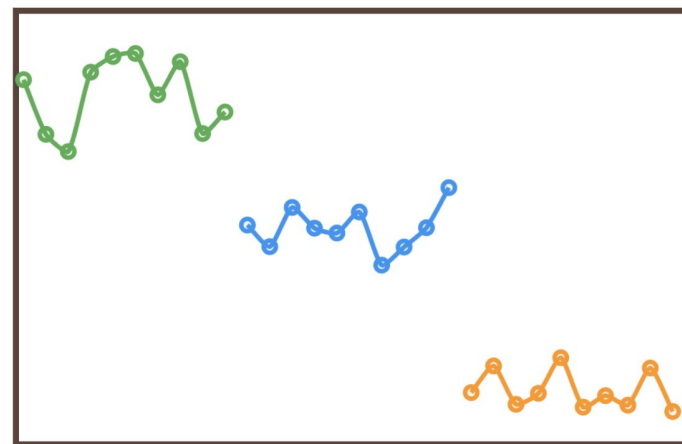
고차원 시계열 데이터를 시간 순서 그대로 입력



비효율적이며 예측 성능 저하

O After : 상태(State) 추상화

공정 상태(State A/B/C)로 분할 후 요약 통계(평균, 분산 등) 사용



590개의 feature를 수십 개로 대폭 축소하여 성능 향상

고차원 시계열 → State Abstraction

미세 정보보다 결과 예측에 집중하여 핵심을 잡다

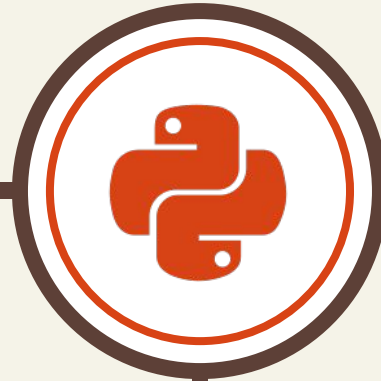
Python으로 수렴하는 문제 정의

바이오 (Bio)

유전자를 개별로 쓰지 않고,
Pathway 단위로 재정의

금융 (Finance)

단순 거래 단위의 복잡성에서 벗어나,
기업 수준의 행동 패턴으로 집약



제조 (Manufacturing)

방대한 고차원 시계열을 그대로 쓰지 않고,
의미 있는 **상태(State)** 요약으로 변환

**좋은 모델은
결과를 개선하지만,
좋은 문제 정의는
방향을 바꿉니다**

—
Python은 단순히 모델을 돌리는 도구가 아니라,
빠르게 시도하고, 실패하고, 다시 재정의할 수 있게 해주는
가장 좋은 가설 검증 도구입니다.



Kyung Seo KIM - LinkedIn